VnV Plan: Pentachess

Version 2.0

Team 6
Brandon Bosman
Mian Rafay
Karl Sader
Saad Tariq
Emaan Khan

Table 1: Revision History

Date	Developer(s)	Change
February 7, 2024	Entire Team	Initial Draft
April 4, 2025	Emaan Khan, Saad Tariq	Final Draft

1. Project Description

This project aims to make Pentachess, a challenging and interesting twist on the original game of chess, more accessible to the public. Because of the complexity of the rules, it would be very difficult to play as a board game, but we believe it is the perfect application for a computer game. Not only will a computer game automate difficult rules and present them in a clean GUI, but it will also allow for new ways to play, such as online matches.

2. Component Test Plan

2.1 Tests for Functional Requirements

2.1.1 Frontend

Game Component

In the game component, we need to test our project on the various possible actions within the game. Our game has important procedures and a unique set of actions within our game reducer that take in a state and payload and return some updated state.

- Ensure that utility functions can handle different edge cases and produce their intended output:
  - **`createGameState`**: Should create the game state with the correct values
  - **`createMove`**: Should create moves with correct values & properly format its notation
  - **`displayTimeRemaining`**: Should format time correctly
  - **`getNewStatus`**: Should update status correctly for each game condition
  - **`isGameOver`**: Should correctly identify game-over statuses
  - **`moveHelper`**: Should correctly handle basic moves and pawn promotion
- Ensure that all game reducer actions update the state correctly:
  - **START_GAME**: Should update the game state correctly when passed/not passed parameters regarding duration and players
  - **SET_STATUS**: Should update the game state correctly for all possible statuses
  - **PROMOTE_PAWN**: Should update the game state correctly when pawn promotion coordinates are set and not set
  - **DECREMENT_TIMER**: Should update the game state correctly for the appropriate player regarding whether the timer is set
  - **SET_WINNER**: Should update the game state correctly when the player has won
  - **END_GAME**: Should update the game state correctly for when the opponent has lost, there's been a draw, or a player has won
  - **RESET_GAME**: Should correctly reset the game state to its initial state
  - **SYNC_GAME**: Should correctly synchronize the game state with the provided state
  - **SET_SELECTED_CELL**: Should correctly update game status when a cell has been selected with the available and invalid moves, as well as when the selected cell is null

    - **SET_PENDING_MOVE**: Should correctly update the state temporarily when a player has set a pending capturing and non-capturing move
    - **CANCEL_MOVE**: Should correctly update the state to revert the pending move and reset the board to the state before the player clicked on the cell
    - **CONFIRM_MOVE**: Should correctly update the state (updates board, previous moves, and captured moves) to confirm the pending move
- Ensure that all in-game divs are displayed after game reducer actions:
    - Should display all required game components (Captured Pieces, Board, Moves, Timer) when the **START_GAME** action is executed
    - Should disable the board and display the End Game modal when the **END_GAME** action is executed
    - Should move a piece in the correct sequence when the following actions (**SET_PENDING_MOVE**, **CANCEL_MOVE**, & **CONFIRM_MOVE**) are executed
    - Should display the Pawn Promotion modal when the **PROMOTE_PAWN** action is executed

Each action is extremely important for the game to work flawlessly; otherwise, the logic will be flawed and inconsistent with what the users were promised.

Board Component

The Board component organizes and renders the board as sides composed of individual cells. We need to test various aspects of the board, including rendering each cell correctly with the correct metadata, ensuring the positioning of all sides is accurate and being shown, and ensuring all utility functions are producing their intended output.

- Ensure that utility functions can handle different edge cases and produce their intended output:
    - **`cloneBoard`**: Should clone a board correctly
    - **`getKingCell`**: Should be able to find the king's cell position if it exists
    - **`getSides`**: Should divide the array into sides with the specified side length
    - **`createBoard`**: Should initialize a board with the correct structure and properties
- Ensure that the board div displays correct metadata and is positioned correctly:
    - Cells are highlighted with the correct color on click action
    - All rings are rendered with correct positioning and number of sides (10 sides, ranging from 0-9)
    - All sides are rendered with correct positioning, ID, and style

Cell & Piece Components

- Ensure that utility functions can handle different edge cases and produce their intended output:
    - **`cellId`**: Should generate correct cell IDs
    - **`cloneCell`**: Should clone a cell correctly
    - **`getCCWEdge`**: Should get the clockwise edge for cells
    - **`getCWEdge`**: Should get the counter-clockwise edge for cells
    - **`getSideEdge`**: Should get the correct edges for cells
    - **`createCell`**: Should create any cell with the correct properties
    - **`setCellEdges`**: Should set cell edges correctly
    - **`setCellVertices`**: Should set cell vertices correctly
    - **`createPiece`**: Should create any piece with its correct properties
- Ensure that pieces are displayed correctly:
    - Each cell is rendered with the correct ID, rotation, and positioning
    - Each piece is rendered on the board with the correct image

- Each piece is rendered on the correct cell, with the correct image on initial setup

Authentication Components
In this section, we need to test our project on the implementation of its functional requirements of authentication for the login, register, and forgot password components. In general, we will need to test the schema for valid and invalid inputs for each page's requirements. For all the associated components, we need to ensure that:

- Email Validation: Should accept valid email addresses and reject invalid ones
- Username Validation: Should accept valid usernames and reject invalid ones
- Password Validation: Should accept valid passwords and reject invalid ones
- *Login* Data Validation: Should accept valid *login* data and reject invalid login data
- *Registration* Data Validation: Should accept valid *registration* data and reject invalid ones
- *Forgot Password* Data Validation: Should accept valid *forgot password* data and reject invalid ones

User Components
In this section, we need to test our project on the implementation of its functional requirements of user components. In general, we will need to test the schema for valid and invalid inputs, and for all the associated components, we need to ensure that:

- Email Validation: Should accept valid email addresses and reject invalid ones
- Username Validation: Should accept valid usernames and reject invalid ones
- Name Validation: Should accept valid names and reject invalid ones
- Password Validation: Should accept valid passwords and reject invalid ones

2.1.2 Backend

Move Logic
In this section, we need to test our project on the implementation of its legal move functionality. There is quite a lot that goes into this:

- **Piece Logic**: Our game consists of 7 different piece types, which all have a different kind of move-set. Some pieces can only passively move to certain locations, while some can only move to certain locations by capturing an enemy piece.
  - It is critical to test **`getInvalidMoves`** and **`getPossibleMoves`** to ensure every piece calculates only the legal available moves from a variety of different starting cells on the board. Ultimately, the board consists of 9 different types of cells, so testing each piece on all 9 types of cells is a priority, as we can't test every single possible combination (it is an unfathomable amount for a game like this).
  - With the addition of the pawn promotion feature in the game, we must also test **`canPromote`** to ensure that only specified pieces within their legal positions qualify for a promotion. As this is a powerful move, any errors can lead to drastic differences in gameplay.
- **Check/Checkmate/Stalemate**: In addition, we also have check/checkmate, which handles the logic of limiting a player's moves and declaring a winner (ending the game). Similarly, stalemate checks to see if a player has no legal moves and their king is not in check, which ends the game in a draw.
  - We need to test **`checkForCheckOrMate`** and **`checkForStalemate`** to ensure that the available moves of a piece are modified so that all moves that put/keep your king in check are disregarded. This way, only moves that put/keep your king out of check are included.

Also, we need to ensure that both functions can detect when these conditions occur so that the game state can be updated.

- **Other Board Conditions**: Finally, we also have a few other conditions that check to see whether the players have made 50+ moves and have yet to capture, repeated a move 3+ times, and do not have enough pieces or a type of piece to capture. These conditions dictate the game state and result in a draw.
    - We need to test **`checkFiftyMoveNoCapature`**, **`checkThreeMoveRepition`**, and **`checkInsufficientMaterial`** to ensure that players can make legal moves in which they capture and, in general, ensure that players are constantly progressing through the game.

WebSocket Events Online (Expected Cases)
We will be using WebSockets to enable online play between two players. Here are the client-to-server WebSocket events: [JOIN], [MOVE], and [END]. Here are the server-to-client WebSocket events: [START], [MOVE], [END], and [LEAVE].

Join a new game:

- [JOIN]: Should put a new player in the game queue
- [JOIN]: Should not put a player in the game queue if the player is already in a game
- [START]: Should start a game between two players when they join the game queue

Make a move:

- [MOVE]: Should sync moves between players in a game

End a game:

- [END]: Should sync end-game data between players in a game and remove the game from the game queue
- [LEAVE]: Should notify the other player that the current player has disconnected and should remove the game from the game queue

2.2 Tests for Non-Functional Requirements

2.2.1 Frontend

Board Component
We must minimize latency within the Board component to deliver a smooth and interactive user experience. There is a lot of metadata that is transferred and updated between the local state and what the user sees. We will ensure that the board correctly updates and loads after each move.

- Ensure that the entire board is loaded after any update, which should take less than 40 milliseconds.
- Ensure that the board is sending dispatches to the local state within 5 milliseconds, and these dispatches include **SET_SELECTED_CELL**, **SET_PENDING_MOVE**, **CANCEL_MOVE**, & **CONFIRM_MOVE.**

Game Component
Looking at the performance of the game component, we need to ensure that the speed of each of the defined game actions in the game reducer is done at a fast pace. Anything higher than 250 milliseconds of loading time would affect the playability, especially since the game is on a timer, and the occurrence of the timer reaching zero results in a loss. Our primary states mentioned above are: **START_GAME**, **SET_STATUS**, **PROMOTE_PAWN**, **DECREMENT_TIMER**, **SET_WINNER**, **END_GAME**, **RESET_GAME** &

SYNC_GAME. Naturally, some actions are run more frequently than others, which is why we need to prioritize the optimization of certain ones over others, as below:

1. **START_GAME** & **END_GAME**: These are vital actions that dictate the general user experience when the user starts and ends a game. The **START_GAME** action is resource-intensive as it initializes the game and board state.
2. **PROMOTE_PAWN**: This is an important action within the game, and we must ensure that it is being handled in a relatively quick manner, with the player's timer being dependent on the speed of their actions in the game.
3. **RESET_GAME** & **SYNC_GAME**: Similar to **START_GAME**, these actions transfer the game state, which is a data-intensive operation. We must ensure that these operations have low execution times.

Overall, as this is an online game, maximizing the game's performance is essential to minimizing load times for any user action.

2.2.2 Backend

1. **Scalability Testing**

   Goal: For this metric, our team aims to evaluate the system's ability to scale as the number of players active on the application increases, resulting in simultaneous games running on the server.

   Test: Gradually increase the number of simulations for simultaneous games being played, representing realistic numbers. Since the application is just starting off, we expect, at maximum, a few hundred games to be played, so we will measure game latency, throughput, and error rates for simulations running for the stated set: {25, 50, 150, 250, 350}. To measure the throughput, we will test any moves transmitted to the opponent and the possible moves they are allowed to make. Error rates will be calculated as an average error per game by taking the total errors that occurred and dividing by the total number of games played. Our goal will be to minimize this to near 0 as these will highly impact the quality of the games. The errors include disconnections, request timeouts, black hole states, etc. We will further stress-test the system's behavior when all simulations make movements simultaneously to monitor any abnormalities that could lead to potential crashes.

2. **Fault Tolerance**

   Goal: This metric's prime objective is to ensure that the system recovers well from any failures users may encounter.

   Test: Our team will simulate potential failures, such as server crashes and network failures, while monitoring the system's behavior. Various tests, including single- and double-ended failures, will be involved.

3. **Optimal Algorithms**

   Goal: Ensure minimal stress on the CPU and optimize any delays in computation

Test: For this metric, our team will review the time it takes to run individual algorithms such as ‘check for check’, ‘calculate possible moves’, ‘checkmate’, etc. The individual algorithms will be given the appropriate data, and the time for each piece type will be recorded with various numbers of pieces on the board. Our team will further manually analyze the algorithms to reduce any redundancy and apply concepts like memoization and dynamic programming where possible, which will contribute to the overall efficiency of the application.